



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

From Characters to Tokens

CMSC 124 Activity 3

Prepared by: Rene Andre Bedonia Jocsing

Published: September 02, 2026

Deadline: September 03, 2026

A **compiler** translates source code into another form. It meets characters first, and turns them into a token stream that the parser can trust. After each token it emits, the scanner leaves its cursor on the next unread character. It skips nothing.

The scanner comes first in this course because it comes first in the compiler, and because Laboratory Activity 1 is implementing it in code, released on September 6. Later activities build the **parser**, which checks whether a token stream forms a legal phrase, and the **semantic analyzer**, which checks the facts that structure alone leaves open, such as whether an identifier was declared before its first use. Today you need only know that both of them read what the scanner hands over.

The language here is invented for the activity. It's written to be read. Every rule it depends on is on this page.

You have 60 minutes for the two checkpoints. Keep each worked example open while you attempt the checkpoint that follows it. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 10 points. Six points reward constructions, and four reward explanations.

The Scanner's Contract

A **lexeme** is the character sequence copied from the source, such as `limit` or `25`. A **token type** is its category, such as `IDENTIFIER` or `NUMBER`. A token record stores both, plus a **source position**, the line and column where the lexeme appears. The **cursor** is the position of the next unread character. A **span** is the inclusive range occupied by one lexeme.

We'll use one-based positions. A span `1:10-14` covers columns 10 through 14 on line 1. `EOF` sits immediately after the last character. It occupies no characters of its own, so both ends of its span sit on that same column. For a line 12 characters long, the span is `1:13-13`.

Read `1:10-14` as "line one, columns ten through fourteen." Read `EOF` as "end of file."

Those five terms describe one journey. The three figures below take the identifier `total` through it, from loose characters to the record the parser reads.

One token record, built from characters

The same five characters, first as source text, then as one lexeme, then as the record the parser receives.

Start with characters

The source holds five separate characters at columns 10 through 14. None of them means anything on its own.

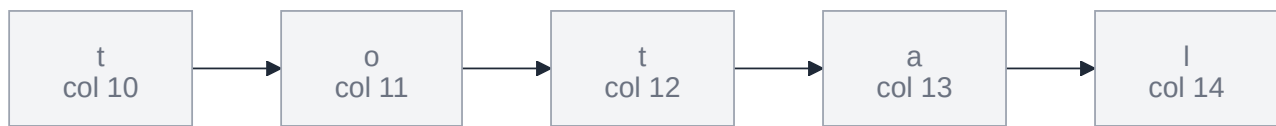


Figure 1: Start with characters

Group them into one lexeme

A letter starts an identifier and letters continue it, so all five characters join one lexeme.

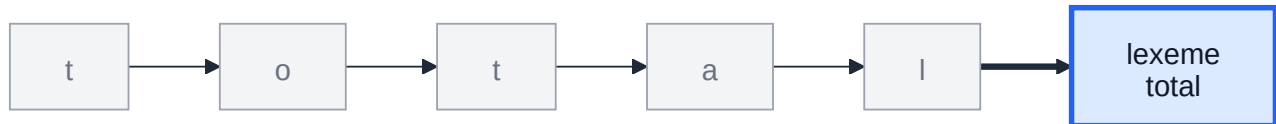


Figure 2: Group them into one lexeme

Store the token record

The record keeps the category, the copied characters, and the place they came from. An identifier also takes a symbol-table entry.

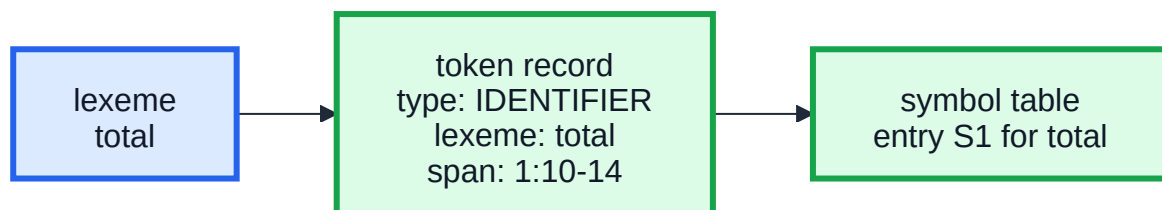


Figure 3: Store the token record

Use these recognition rules:

First character	Continue while	Token type
A letter or _	Letter, digit, or _	IDENTIFIER, unless the whole lexeme is the keyword <code>while</code> , which produces <code>WHILE</code>
A digit	Digit	<code>NUMBER</code>
<	Take a following = when present	<code>LESS_EQUAL</code> for <=, otherwise <code>LESS</code>
=	No continuation in today's language	<code>EQUAL</code>
;	No continuation	<code>SEMICOLON</code>
(or)	No continuation	<code>LEFT_PAREN</code> or <code>RIGHT_PAREN</code>

Spaces and line breaks separate lexemes but produce no token. Any other character produces a scanner error at its position.

The scanner uses **longest-prefix recognition**. It takes the longest characters from the current position that form one token. **Lookahead** inspects the next character before deciding whether it belongs to the current lexeme, as when distinguishing < from <=.

Read `<=` as “less than or equal.” Yep.

The **symbol table** records each identifier once. For this activity, give identifiers IDs in order of first appearance: S1, S2, and so on. Keywords, punctuation, and numbers don’t enter the table.

Worked Example for Checkpoint 1

Suggested time: 30 minutes, including the worked example.

Scan `while (n<7);`. The spaces separate lexemes but produce no tokens.

Step	Token type	Lexeme	Span	Cursor after token
1	WHILE	while	1:1-5	1:6
2	LEFT_PAREN	(	1:7-7	1:8
3	IDENTIFIER	n	1:8-8	1:9
4	LESS	<	1:9-9	1:10
5	NUMBER	7	1:10-10	1:11
6	RIGHT_PAREN	)	1:11-11	1:12
7	SEMICOLON	;	1:12-12	1:13
8	EOF	empty	1:13-13	end

The symbol table contains only `S1 → n`. Read that notation as “symbol-table entry S1 refers to n.” At `<`, lookahead sees `7`, so the scanner emits `LESS` and leaves `7` unread. Longest-prefix recognition chooses the longest prefix that forms one complete token. Later tokens may not extend that prefix. This is the **maximal munch** principle.

Splitting a source that begins with `while2` into `WHILE` followed by the identifier `2` is wrong. That fails because an identifier may continue through digits. The scanner must take all of `while2` first and then classify the complete lexeme. It isn’t the keyword `while`.

The four figures below are one sequence. The characters never move. What changes is how much of the line has been classified, and where the cursor sits.

The scanner cursor moves forward

Blue marks the token just recognized. Gray characters are still unread.

Recognize the keyword

The scanner takes the longest identifier-shaped prefix, then classifies the complete lexeme `while` as a keyword.



Figure 4: Recognize the keyword

Record the identifier

After the parenthesis, `n` becomes one identifier and receives symbol-table entry S1.



Figure 5: Record the identifier

Use lookahead at the less-than sign

The next character is 7, not =, so the scanner emits LESS without consuming the digit.



Figure 6: Use lookahead at the less-than sign

Finish the token stream

The digit, closing parenthesis, and semicolon become separate tokens. EOF sits at column 13.



Figure 7: Finish the token stream

Checkpoint 1: Build the Stream (5 Points)

Running Case

Scan this complete source line:

```
1 while2 = limit<=25;
```

Start with the cursor at line 1, column 1. After emitting a token, move the cursor to the first unread character, even when that character is a space. Don't discard a character just because its role looks obvious.

Copy and complete this table through EOF.

Step	Token type	Lexeme	Span	Cursor after token
1				
2	EQUAL	=		
3				
4				
5	NUMBER	25		
6				
7	EOF	empty		end

Then draw the symbol table after the scan.

1. (Construction: 3 points) Submit the complete token and symbol-table records.

2. (Explanation: 2 points) Explain why `while2` isn't the keyword `while`, and identify the exact decision that requires lookahead. Cite the longest-prefix rule in your explanation. For each half, cite the rule and the character that triggers it. A sentence each is enough.

Where the Scanner Stops

The scanner recognizes lexemes. The **parser** receives the completed token stream and asks whether those tokens form a legal phrase. The **semantic analyzer** checks context-sensitive facts after structure

is known, such as whether a used identifier was declared. These boundaries matter because one line can contain errors from several phases.

For example, `x = ;` contains only recognizable characters, so scanning succeeds. A parser must reject the missing expression. In contrast, `x = @;` fails during scanning because `@` matches no token rule.

Worked Example for Checkpoint 2

Suggested time: 30 minutes, including the worked example.

Compare `x = ;` with `x = @;`. The first source becomes IDENTIFIER EQUAL SEMICOLON EOF, so the scanner finishes and the parser reports the missing expression. The second source reaches `@` after emitting IDENTIFIER and EQUAL. No scanner rule accepts that character, so parsing never begins.

Source	Last scanner result	Reporting phase
<code>x = ;</code>	Complete token stream	Parser
<code>x = @;</code>	Error at <code>@</code>	Scanner

Both lines look like incomplete assignments, but the scanner's contract still applies. The `@` prevents the scanner from producing the token stream a parser would need, while the incomplete assignment produces an error in the parser itself.

Checkpoint 2: Transfer the Contract (5 Points)

Now scan this fresh source. Spaces still move the cursor forward and still emit no token. Stop only when a rule tells you to stop.

```
1 while (count1 <= 25) @
```

1. (Construction: 3 points) Write every token record that can be emitted before the first error. Include each lexeme and span. Then record the error character and its position.

2. (Explanation: 2 points) Answer these in order. A sentence each is enough. Cite the rule behind every answer.

- Why is `count1` one token and not two?
- Why is `<=` one token and not two?
- Which compiler phase reports the `@`?
- Suppose the `@` were deleted and the line ended after `)`. Which later phase would reject the missing loop body, and why?